UNIVERSAL PROMPT ENGINEERING GUIDELINE

For GPT-5.6 Luna and Gemini 3.8 Flash

A comprehensive framework for high-quality, accurate, efficient code generation

PART 1 — THE SIX CORE JUDGING CRITERIA

Every prompt and generated solution must be evaluated against these six criteria. They are not independent; a weakness in one can affect the quality of the entire solution.

1. Prompt Quality & Structure

The prompt must communicate the task clearly, logically, and without ambiguity.

A high-quality prompt should:

- Clearly identify the problem that needs to be solved.
- Define the exact expected outcome.
- Provide all relevant background information.
- Specify the programming language, frameworks, libraries, and environment.
- State functional and non-functional requirements.
- Identify restrictions and things that must not be changed.
- Explain the expected output and its format.
- Avoid contradictory, repetitive, vague, or unnecessary instructions.

Judging standard: Could a competent developer understand exactly what needs to be built without guessing?

2. Accuracy & Relevance of Output

The generated solution must address the actual problem rather than produce something that merely looks relevant.

It should:

- Follow every mandatory requirement.
- Use technically correct logic.
- Avoid fabricated information, APIs, libraries, functions, and features.

- Use compatible versions and valid syntax.
- Preserve the user's existing requirements and data.
- Clearly identify assumptions and missing information.
- Distinguish verified facts from recommendations.

Judging standard: Is the output technically correct and directly relevant to the requested task?

3. Creativity

Creativity means finding useful, practical, and sometimes unconventional ways to improve the solution without compromising requirements.

The AI should:

- Explore better approaches when appropriate.
- Identify opportunities to improve functionality.
- Suggest useful features that genuinely support the objective.
- Consider alternative algorithms and architectures.
- Improve usability and user experience.
- Avoid unnecessary complexity and decorative features that add no value.

Important: Creativity must never override mandatory requirements. Additional ideas must be clearly distinguished from requested functionality.

Judging standard: Does the solution demonstrate thoughtful innovation while remaining practical?

4. Problem Solving

The AI must understand the underlying problem before implementing a solution.

It should:

- Identify the root cause, not merely the visible symptoms.
- Break complex problems into manageable components.
- Understand dependencies between components.
- Evaluate alternative approaches.
- Consider edge cases and failure scenarios.
- Handle invalid inputs and unexpected situations.
- Choose a solution that addresses the complete problem.

Judging standard: Does the solution genuinely solve the problem, including the difficult cases?

5. Efficiency & Optimization

The solution should achieve the required outcome without unnecessary resource consumption or complexity.

Consider:

- Time complexity and space complexity.
- Database query efficiency.
- Memory consumption.
- Network requests and API usage.
- Loading time and response time.
- Code duplication.
- Unnecessary dependencies.
- Scalability and maintainability.
- Efficient algorithms and data structures.

Do not optimize prematurely. Correctness, security, and maintainability come first. Optimize measured or reasonably foreseeable bottlenecks.

Judging standard: Is the solution efficient without sacrificing correctness or readability?

6. Final Output Quality

The final result must be complete, usable, and properly presented.

For coding tasks:

- Code must be complete, not an unexplained collection of fragments.
- File names and directory structures must be clear.
- Dependencies and installation instructions must be included.
- Setup and execution instructions must work.
- Database requirements and configuration must be explained.
- Testing results must be reported honestly.
- Known limitations must be disclosed.
- The final implementation must match the original requirements.

Judging standard: Can the user take the result and use it successfully in the intended environment?

PART 2 — FUNDAMENTAL PRINCIPLES OF PROMPT ENGINEERING

1. Define the outcome before the process

Begin by explaining what you want the AI to accomplish.

A good prompt describes:

- What needs to be built.
- Why it is needed.
- Who will use it.
- What successful completion looks like.

Avoid forcing the AI to follow dozens of unnecessary steps when it can choose a better implementation approach.

However, explicitly define steps that are essential for correctness, safety, or compatibility.

2. Provide sufficient context

Never expect the AI to understand information that has not been provided or cannot be accessed.

Supply relevant:

- Project background.
- Problem statements.
- Existing source code.
- Screenshots and UI references.
- Database schemas.
- API documentation.
- Error messages and logs.
- Technical specifications.
- Previous implementation decisions.
- Existing limitations.

When providing files, explain what each file contains and why it matters.

For large projects, provide the project structure and relevant files before asking for implementation.

3. Use clear and consistent sections

Organize prompts using headings or XML-style tags.

A useful structure is:

```
# ROLE
# PROJECT CONTEXT
# OBJECTIVE
# FUNCTIONAL REQUIREMENTS
# TECHNICAL REQUIREMENTS
# CONSTRAINTS
# EXPECTED BEHAVIOUR
```

```
# EDGE CASES
# SUCCESS CRITERIA
# VALIDATION REQUIREMENTS
# OUTPUT FORMAT
```

Choose Markdown headings or XML-style tags and use them consistently. Avoid mixing multiple complicated organizational systems without a reason.

4. State every important instruction clearly

Avoid vague instructions such as:

- Make it perfect.
- Make it advanced.
- Make it professional.
- Make it optimized.
- Make it user-friendly.

Replace them with measurable requirements.

For example:

Instead of:

"Make the website responsive."

Write:

"The website must work on mobile phones, tablets, laptops, and desktop screens. Navigation, tables, forms, buttons, and content must remain usable without horizontal overflow."

5. Establish priorities

Not all requirements have equal importance.

Specify the priority order:

1. Correctness and mandatory requirements.
2. Data integrity and security.
3. Functional completeness.
4. Compatibility with the existing environment.
5. Performance and optimization.
6. Maintainability and code quality.
7. Visual polish and optional improvements.

This prevents the AI from prioritizing attractive design over essential functionality.

6. Eliminate conflicting instructions

Before submitting a prompt, check whether two instructions contradict each other.

Examples:

- Requesting a lightweight application while demanding many unnecessary dependencies.
- Requesting complete code while prohibiting changes to every relevant file.
- Requiring real-time data while prohibiting all external connections.
- Requesting three recommendations while requiring every recommendation to be mutually exclusive when only two valid options exist.

When requirements conflict, instruct the AI to identify the conflict and ask for clarification rather than silently choosing one.

7. Use examples when behaviour is difficult to describe

Examples are especially useful for:

- Input and output formats.
- Database records.
- Expected calculations.
- API responses.
- UI behaviour.
- Validation rules.
- Sorting and ranking.
- Error handling.

A good example should demonstrate the expected behaviour, not merely show unrelated sample data.

PART 3 — THE COMPLETE CODE-GENERATION FRAMEWORK

This is the most important section. Use it whenever you want an AI model to build, modify, debug, or optimize software.

1. Define the role

Give the AI a role appropriate to the task.

Example:

"You are a senior software engineer, system architect, database designer, cybersecurity reviewer, and quality assurance engineer. You are responsible for producing a complete, secure, maintainable, and tested implementation."

The role should establish expertise and responsibilities, not just sound impressive.

2. Explain the project

Describe the project in sufficient detail.

Include:

- Project name.
- Purpose.
- Problem being solved.
- Target users.
- Main use cases.
- Existing functionality.
- Current development stage.
- Expected final product.

The AI must understand the overall system before making changes to individual components.

3. Define the exact objective

State precisely what you want.

Examples:

- Build a complete web application.
- Implement a new authentication system.
- Fix a specific database error.
- Optimize an existing algorithm.
- Add a feature without modifying existing functionality.
- Refactor a module while preserving its behaviour.

Avoid combining unrelated objectives into one unclear request.

4. Provide detailed functional requirements

Functional requirements describe what the software must do.

For every feature, specify:

A. Input

- What information does the user provide?
- What is the input format?
- Which fields are mandatory?
- What validation is required?

B. Processing

- What should happen to the input?
- What calculations or logic are required?
- What rules must be followed?
- What conditions affect the result?

C. Output

- What should the system display, store, return, or generate?
- What should happen when the operation succeeds?
- What should happen when it fails?

D. User interaction

- What actions can users perform?
- What happens after each action?
- What should the interface display?
- What feedback should users receive?

Each feature should have an identifiable expected behaviour.

5. Specify the technical environment

This is essential for generating executable code.

Provide:

Category	Information to specify
Language	Python, Java, JavaScript, C++, etc.
Version	Exact language/runtime version when relevant
Frontend	HTML, CSS, JavaScript, React, etc.
Backend	Flask, Django, Spring Boot, etc.
Database	SQLite, PostgreSQL, MySQL, etc.
Libraries	Required and permitted dependencies
Operating system	Windows, Linux, macOS, etc.
IDE	VS Code, IntelliJ, PyCharm, etc.
Deployment	Localhost, cloud platform, server, etc.
API	Required external or internal APIs
Existing project	Current folder and file structure

Never let the AI assume a technology stack when the stack is already specified.

If the stack is not specified, ask it to recommend a suitable one based on the project's actual requirements.

6. Define architecture and project structure

For larger projects, explain how the code should be organized.

Specify:

- Frontend and backend separation.
- Database access layer.
- Business logic layer.
- API routes.
- Authentication and authorization.
- Configuration management.
- Utility functions.
- Static files and templates.
- Error handling.
- Logging.
- Testing structure.

Ask the AI to choose a simple architecture that is appropriate for the project's size.

Avoid unnecessary architectural patterns for small projects.

7. Define database requirements

Database mistakes can cause serious problems.

Specify:

- Tables and their purpose.
- Column names and data types.
- Primary keys.
- Foreign keys.
- Unique constraints.
- Relationships.
- Required and optional fields.
- Default values.
- Indexes.
- Data validation.
- Deletion and update behaviour.
- Migration requirements.

For an existing database, explicitly instruct the AI to inspect the current schema before changing it.

Critical data-integrity rule:

Never delete, overwrite, reset, truncate, or modify existing user data unless explicitly authorized. Preserve existing records and relationships. Use safe migrations and backups when schema changes are necessary.

8. Define API and integration requirements

If the application uses APIs, specify:

- API provider.
- Endpoint.
- HTTP method.
- Authentication method.
- Request parameters.
- Request body.
- Expected response.
- Error codes.
- Rate limits.
- Timeout requirements.
- Retry behaviour.
- Data format.

If documentation is available, provide it.

Do not allow the AI to invent endpoints, parameters, API keys, or response structures.

If an API is unfamiliar or may have changed, instruct the AI to verify it against current official documentation.

9. Define UI/UX requirements

For frontend projects, describe the expected interface.

Include:

- Layout.
- Navigation.
- Page structure.
- Typography.
- Colours.
- Spacing.
- Components.
- Forms.
- Buttons.
- Tables.
- Charts.
- Responsive behaviour.
- Accessibility.
- Loading states.
- Empty states.
- Error states.
- Success messages.

When providing screenshots or reference designs, explain which elements must be reproduced and which can be improved.

Do not assume that a screenshot communicates every interaction or responsive requirement.

10. Define constraints and restrictions

Explicitly state what the AI must preserve or avoid.

Examples:

- Do not change the existing database schema.
- Do not remove existing features.
- Do not introduce unnecessary dependencies.
- Do not change existing API contracts.
- Do not use cloud services.
- Do not replace the existing framework.
- Do not fabricate test results.
- Do not expose secrets or credentials.
- Do not rewrite unrelated files.

Distinguish between:

- Mandatory restrictions.
- Preferences.
- Optional recommendations.

This distinction prevents the AI from treating a preference as an absolute requirement or ignoring a hard restriction.

11. Define expected behaviour and edge cases

A good implementation must work beyond the ideal scenario.

For every important feature, consider:

Input edge cases

- Empty input.
- Missing fields.
- Incorrect data types.
- Invalid formats.
- Extremely large values.
- Duplicate entries.
- Special characters.

Application edge cases

- No records available.
- Multiple simultaneous users.
- Network failure.
- Database connection failure.

- API timeout.
- Invalid authentication.
- Expired sessions.
- Repeated submissions.
- Unexpected application restart.

Data edge cases

- Duplicate records.
- Null values.
- Inconsistent relationships.
- Invalid references.
- Concurrent updates.
- Partial transactions.

Ask the AI to identify additional realistic edge cases that are relevant to the project.

Do not force irrelevant edge cases into every small program.

12. Specify security requirements

Security must be considered during design, not added as an afterthought.

For relevant applications, require:

- Input validation and sanitization.
- Parameterized database queries.
- Secure password hashing.
- Authentication and authorization.
- Session security.
- Protection against SQL injection.
- Protection against cross-site scripting.
- CSRF protection where applicable.
- Secure secret management.
- Safe file uploads.
- Appropriate access controls.
- Rate limiting where necessary.
- No hardcoded production credentials.

The AI should explain significant security risks and avoid claiming that an application is completely secure.

13. Specify performance and scalability

Explain the expected workload.

Examples:

- Approximate number of users.

- Expected database size.
- Number of concurrent requests.
- Maximum file size.
- Response-time expectations.
- Available memory and storage.
- Expected traffic growth.

Ask the AI to identify likely bottlenecks and optimize only where justified.

For database-heavy systems, examine query count, indexing, pagination, transactions, and unnecessary repeated queries.

14. Define success criteria before coding

This is one of the most effective ways to improve code-generation accuracy.

Write testable conditions that determine whether the implementation is complete.

Example:

"The task is complete only when:

- All required features work.
- Existing functionality remains intact.
- Data is stored correctly.
- Invalid input is handled safely.
- The application starts successfully.
- All relevant tests pass.
- No mandatory requirement is missing.
- Setup instructions are provided."

A feature is not complete merely because the AI has written code for it.

PART 4 — THE REQUIRED IMPLEMENTATION WORKFLOW

For substantial coding tasks, ask the AI to follow this workflow. The aim is not to force unnecessary steps, but to ensure that implementation decisions are grounded in the actual project.

Stage 1: Understand

Before coding:

1. Read the complete problem statement.

2. Inspect the relevant existing files.
3. Understand the project structure.
4. Identify the current architecture.
5. Understand existing functionality.
6. Identify dependencies and version requirements.
7. Identify constraints and risks.
8. Identify missing information that materially affects implementation.

Do not modify files before understanding the relevant existing implementation.

Stage 2: Plan

Develop a concise implementation plan.

Identify:

- Files that need to be created.
- Files that need to be modified.
- Existing components that can be reused.
- Database changes.
- Dependencies.
- Important logic.
- Integration points.
- Potential failure scenarios.
- Validation methods.

For small, straightforward tasks, the plan can be brief.

Stage 3: Evaluate the solution

Before implementation, consider:

- Is the proposed approach suitable?
- Is there a simpler solution?
- Does it satisfy every mandatory requirement?
- Is it compatible with the existing project?
- Will it preserve existing functionality?
- Does it introduce unnecessary dependencies?
- Are there security or data-integrity risks?

Do not over-engineer a simple problem.

Stage 4: Implement

Generate the solution using:

- Clear naming conventions.
- Modular code where appropriate.

- Reusable functions.
- Appropriate data structures.
- Consistent formatting.
- Meaningful error handling.
- Minimal necessary dependencies.
- Proper separation of concerns.

Do not introduce unrelated changes.

Stage 5: Integrate

Check that all components work together.

For example:

- Frontend communicates correctly with backend.
- Backend communicates correctly with database.
- API responses are processed correctly.
- Forms send the expected data.
- Authentication works across relevant pages.
- Database updates appear correctly in the interface.
- Existing features still function.

A collection of individually correct files does not necessarily form a working application.

Stage 6: Test and verify

Run appropriate checks.

The AI should use available tools and the actual project environment wherever possible.

Tests should include:

1. Syntax validation.
2. Import and dependency checks.
3. Unit tests.
4. Integration tests.
5. Database tests.
6. Input validation tests.
7. Edge-case tests.
8. Security checks where relevant.
9. Build checks.
10. End-to-end or smoke tests where feasible.

Run the most relevant tests first. A full test suite is preferable when practical.

Stage 7: Fix and retest

When an error occurs:

1. Read the actual error message.
2. Identify the underlying cause.
3. Trace the issue to the relevant code.
4. Apply the smallest appropriate fix.
5. Check for unintended side effects.
6. Rerun the failed test.
7. Rerun relevant regression tests.

Do not simply suppress error messages or add workarounds that hide the underlying problem.

Stage 8: Final review

Before declaring completion, compare the final implementation against the original requirements.

Check:

- Missing functionality.
- Incorrect calculations.
- Broken imports.
- Missing files.
- Incompatible dependencies.
- Incorrect database operations.
- Security issues.
- UI inconsistencies.
- Incomplete setup instructions.
- Unintended changes.

Only declare success for checks that were actually performed.

PART 5 — SPECIAL RULES FOR WORKING WITH EXISTING CODE

This section is especially important when modifying an existing project.

1. Inspect before modifying

Never assume how an existing application works.

Read relevant files, identify dependencies, understand function calls, and trace data flow before changing code.

2. Preserve existing functionality

Unless explicitly requested:

- Do not remove existing features.
- Do not rename important functions unnecessarily.
- Do not change database structures unnecessarily.
- Do not alter existing routes or API contracts.
- Do not change existing user data.
- Do not redesign unrelated pages.
- Do not rewrite the entire project when a targeted modification is sufficient.

3. Make minimal, controlled changes

Prefer a targeted fix over a complete rewrite.

If multiple files must change, explain why each change is necessary.

4. Check dependencies

Before introducing a new library:

- Verify that it exists.
- Check compatibility.
- Consider whether it is necessary.
- Check whether the project already has an equivalent dependency.
- Explain installation and configuration requirements.

5. Respect existing conventions

Follow the project's established:

- Naming conventions.
- Folder structure.
- Coding style.
- Error-handling approach.
- Database patterns.
- API design.
- Configuration system.

Do not introduce a completely different coding style without a good reason.

6. Protect existing data

For any database modification:

- Understand the current schema.
- Preserve existing records.
- Avoid destructive migrations.
- Use transactions where appropriate.

- Back up important data before risky changes.
- Verify the result after migration.

7. Report every changed file

At the end, provide:

- File name.
- What changed.
- Why it changed.
- Any new dependencies.
- Any database modifications.
- Any required configuration changes.

PART 6 — ADVANCED TECHNIQUES FOR BETTER PROMPTING

1. Zero-shot prompting

Give the AI a clear task without examples.

Use it when:

- The task is straightforward.
- Requirements are already clear.
- The expected output is conventional.

Example:

"Write a Python function that validates an email address using an appropriate validation approach. Include invalid-input handling and tests."

2. Few-shot prompting

Provide examples of the expected input and output.

Use it when:

- Output formatting must be consistent.
- Business rules are unusual.
- The expected behaviour is difficult to explain.
- A specific coding pattern must be followed.

Example:

```
Input:
{
  "name": "Ali",
  "age": 20
}

Expected output:
{
  "valid": true,
  "message": "User accepted"
}
```

Provide examples that cover normal and unusual cases.

3. Decomposition

Break a complex task into meaningful subproblems.

For example, an application can be divided into:

1. Authentication.
2. Database design.
3. Backend APIs.
4. Frontend interface.
5. Integration.
6. Testing.
7. Deployment.

Use decomposition when it reduces complexity or allows meaningful verification.

4. Comparative reasoning

Ask the AI to compare multiple possible solutions before choosing one.

Example:

"Compare SQLite and PostgreSQL for this application based on concurrency, hosting, maintenance, scalability, and cost. Recommend one option and justify the decision."

Do not ask for alternatives when the technology choice is already mandatory.

5. Assumption management

Tell the AI how to handle missing information.

Recommended rule:

"Do not invent important requirements. If a missing detail materially changes the architecture, functionality, security, or correctness, ask a focused clarification question. For minor details, use a reasonable assumption, state it briefly, and continue."

6. Self-verification

Ask the AI to compare its output against the requirements.

Example:

"Review the implementation against every functional and technical requirement. Identify missing, incomplete, or incorrectly implemented requirements and fix them before finalizing."

Self-review is useful, but it is not a substitute for executing tests.

7. Independent review

For critical work, request a separate review after implementation.

Example:

"Now review the completed code as an independent senior developer who did not write it. Search for logical errors, security vulnerabilities, data-loss risks, incorrect assumptions, performance problems, and missing edge cases."

This encourages a different perspective, although it cannot guarantee that all defects will be found.

8. Iterative refinement

Improve the result through targeted feedback.

Instead of saying:

"Improve the code."

Say:

"The application works, but the database is queried separately for every participant. Refactor this section to reduce unnecessary queries while preserving the exact existing output and behaviour. Add a test to verify the result."

Specific feedback produces more useful revisions.

9. Traceability

For complex projects, ask for a requirement-to-implementation mapping.

Example:

Requirement	Implementation	Verification

User login	Authentication module	Login tests
Save records	Database layer	Database tests
Display results	Results page	UI test
Validate input	Validation functions	Invalid-input tests

This makes missing requirements easier to detect.

PART 7 — MODEL-SPECIFIC OPTIMIZATION

The universal framework remains the same, but the following settings and prompting adjustments are specific to each model.

A. GPT-5.6 Luna

Model identifier: `gpt-5.6-luna`

Recommended prompting approach

GPT-5.6 Luna benefits from prompts that clearly define the intended outcome, relevant context, hard constraints, approval boundaries, and success criteria.

Recommended practices:

1. Use outcome-first instructions.
2. Provide precise project context.
3. State each instruction once.
4. Avoid repetitive and unnecessarily restrictive instructions.
5. Define which actions the AI can take independently.
6. Define when user approval is required.
7. Allow the model to choose an efficient implementation approach within the constraints.
8. Require actual validation when tools and execution environments are available.
9. Specify what must be preserved in existing projects.
10. Request concise explanations of important implementation decisions.

Reasoning effort

For API configurations, GPT-5.6 supports:

Setting	Suggested use
`none`	Simple, low-complexity tasks
`low`	Routine code generation and small fixes

`medium`	General development and debugging
`high`	Complex algorithms and multi-file projects
`xhigh`	Difficult architecture and complex debugging
`max`	Particularly demanding, quality-first tasks

These are practical starting points, not guaranteed performance rankings. Choose based on the task and evaluate the results.

For very difficult tasks, GPT-5.6 also supports a separate `pro` reasoning mode in the Responses API. It is independent of reasoning effort and may increase latency and token usage.

Autonomy and approval boundaries

Use a clear policy:

- Allow reading project files, inspecting logs, editing in-scope code, and running non-destructive tests when implementation is requested.
- Do not make destructive changes, external writes, purchases, or scope-expanding changes without approval.
- If the user asks for an explanation or review, do not implement changes unless requested.
- If implementation is requested, complete the authorized work instead of repeatedly asking permission for ordinary in-scope steps.

Recommended GPT-5.6 Luna instruction

```
Work as an outcome-focused senior software engineer.

Understand the intended result, inspect the relevant project context,
and implement the simplest reliable solution that satisfies all
mandatory requirements.

Preserve existing functionality and data. Avoid unnecessary
rewrites, dependencies, and repeated instructions.

Use available tools to validate the implementation. Fix
identified defects and retest affected functionality.

Do not claim that code works unless the relevant checks have
actually been performed. Clearly report anything that could
not be verified.
```

B. GEMINI 3.8 FLASH

Model identifier: `gemini-3.8-flash`

Recommended prompting approach

Gemini 3.8 Flash responds well to direct, clearly structured prompts with explicit constraints and expected outcomes.

Recommended practices:

1. Use concise and precise instructions.
2. Separate context, task, constraints, and output requirements.
3. Use consistent Markdown headings or XML-style tags.
4. Define ambiguous terms and expected behaviour.
5. Put essential requirements near the beginning.
6. For large codebases or long documents, provide the context first and place the specific task at the end.
7. Explicitly request detailed output when needed.
8. Use examples for complex or unusual requirements.
9. Ask for verification of important implementation decisions.
10. Avoid unnecessary repeated instructions and excessively complicated prompting.

Thinking level

Gemini 3.8 Flash supports:

Setting	Suggested use
`low`	Simple tasks and quick fixes
`medium`	General coding and moderately complex projects
`high`	Complex algorithms, debugging, and multi-step development

`medium` is the default. `minimal` is not supported by Gemini 3.8 Flash.

For substantial coding tasks, medium is a sensible starting point. Use high when the task genuinely requires deeper reasoning, complex planning, or extensive tool use.

Recommended Gemini 3.8 Flash instruction

```
Act as a senior software engineer responsible for producing
a complete and reliable implementation.

Follow the supplied requirements precisely. Use the provided
project context as the source of truth for existing behaviour.

Before modifying code, understand the relevant files,
dependencies, architecture, and data flow.

Implement the requested solution without removing or
changing unrelated functionality.

Check syntax, logic, integration, edge cases, and security.
Run available tests and correct any identified problems.

Clearly distinguish verified results from assumptions
and untested areas.
```

Important distinction

The model-specific instructions above are for text-based code generation and software engineering. Gemini's separate TTS prompting rules should not be mixed into ordinary code-generation prompts.

PART 8 — CROSS-CHECKING AND ERROR PREVENTION

This is a mandatory quality-control section for every substantial coding task.

1. Requirement verification

Create a checklist of all requirements.

For every requirement, determine:

- Is it implemented?
- Is it implemented correctly?
- Does it work with the other components?
- Has it been tested?
- Is there evidence supporting the completion claim?

Never mark a requirement complete merely because related code exists.

2. Syntax verification

Check:

- Syntax errors.
- Missing brackets.
- Incorrect indentation.
- Invalid imports.
- Incorrect variable names.
- Invalid function calls.
- Incorrect data types.
- Unclosed strings.
- Incorrect configuration syntax.

Use the actual compiler, interpreter, linter, or relevant validation tool where possible.

3. Logical verification

Check:

- Conditional statements.
- Loops.

- Mathematical calculations.
- Sorting and filtering.
- Date and time calculations.
- Boundary conditions.
- State transitions.
- Duplicate handling.
- Data consistency.

For critical calculations, use independent examples or test cases to verify the result.

4. Integration verification

Check whether:

- All required files exist.
- Functions are called correctly.
- Imports resolve.
- Routes match frontend requests.
- Database queries match the schema.
- API responses match the expected format.
- Data is passed correctly between components.

5. Security verification

Check for:

- Hardcoded secrets.
- SQL injection.
- Unsafe input handling.
- Missing access controls.
- Exposed sensitive data.
- Unsafe file operations.
- Insecure authentication.
- Improper error disclosure.

6. Regression verification

When modifying existing software, check that previously working functionality still works.

A fix that solves one issue but breaks three existing features is not a successful fix.

7. Output verification

Before submitting the final response:

- Confirm that all requested files are included.
- Check file names and locations.
- Check setup instructions.

- Check dependency lists.
- Check configuration instructions.
- Check that no required code is missing.
- Check that the implementation matches the stated requirements.

8. Honest reporting

Require the AI to distinguish between:

- **Verified:** Actually tested or inspected with evidence.
- **Not verified:** Could not be tested in the available environment.
- **Assumed:** Based on a stated assumption.
- **Recommended:** Suggested improvement, not a mandatory requirement.
- **Blocked:** Cannot be completed without missing information or permissions.

Never accept invented test results, fabricated API responses, or claims of successful execution without evidence.

PART 9 — EFFICIENCY AND OPTIMIZATION RULES

A high-quality prompt should not waste the model's effort.

1. Avoid unnecessary verbosity

Long prompts are not automatically better.

Include all information that affects correctness, but remove:

- Repeated instructions.
- Unrelated background information.
- Unnecessary motivational language.
- Duplicate constraints.
- Irrelevant examples.
- Instructions that do not affect the task.

2. Avoid unnecessary clarification

If the task is sufficiently clear, proceed using reasonable assumptions for minor details.

Ask questions only when missing information could materially change the implementation or create a significant risk.

Ask the smallest number of useful questions.

3. Avoid unnecessary rewrites

For existing projects, prefer minimal, targeted changes.

Do not rewrite an entire application to correct a single function unless the architecture genuinely requires it.

4. Avoid unnecessary dependencies

Use built-in functionality or existing libraries where appropriate.

Introduce new dependencies only when they provide meaningful value.

5. Avoid over-engineering

Do not introduce:

- Unnecessary microservices.
- Excessive abstraction.
- Complicated design patterns.
- Multiple databases without justification.
- Unnecessary caching.
- Unnecessary external APIs.
- Complex deployment systems for simple projects.

6. Optimize based on requirements

A student prototype, a production web application, and an enterprise system have different needs.

Match the implementation to the expected workload, available resources, budget, and maintenance requirements.

PART 10 — MASTER PROMPT TEMPLATE FOR CODE GENERATION

The following is the recommended reusable template. Fill in the relevant sections before submitting it to either model.

```
# ROLE

Act as a senior software engineer, system architect,
database designer, and quality assurance engineer.

Your responsibility is to understand the requirements,
```

produce a complete implementation, and verify the result.

1. PROJECT CONTEXT

Project name:
[Enter project name]

Project purpose:
[Explain what the project does]

Problem being solved:
[Explain the problem]

Target users:
[Identify intended users]

Current development stage:
[New project / Existing project / Debugging / Enhancement]

2. MAIN OBJECTIVE

[Clearly explain exactly what you want the AI to build,
modify, fix, or optimize.]

The final result must achieve:
[Describe the intended outcome.]

3. TECHNICAL ENVIRONMENT

Programming language:
[Specify]

Language/runtime version:
[Specify]

Frontend:
[Specify]

Backend:
[Specify]

Database:
[Specify]

Libraries and frameworks:
[Specify]

Operating system:
[Specify]

Development environment:
[Specify]

Deployment environment:
[Specify]

4. EXISTING PROJECT INFORMATION

Project structure:
[Provide directory structure]

Existing files:
[Provide relevant source code or attachments]

Existing functionality:
[Explain what currently works]

Known problems:
[Explain existing issues]

5. FUNCTIONAL REQUIREMENTS

Requirement 1:
[Describe functionality and expected behaviour]

Requirement 2:
[Describe functionality and expected behaviour]

Requirement 3:
[Describe functionality and expected behaviour]

Add more requirements as necessary.

6. TECHNICAL REQUIREMENTS

Specify:
- Architecture
- Database structure
- API requirements
- Data flow
- Authentication
- Validation
- Error handling
- Integration requirements
- Performance requirements

7. UI/UX REQUIREMENTS

[Describe pages, layout, components, responsiveness,
design references, and expected interactions.]

8. CONSTRAINTS

Mandatory:
[List all non-negotiable requirements]

Restrictions:
[List prohibited technologies, operations, or changes]

Preferences:
[List preferred approaches that are not mandatory]

9. DATA INTEGRITY AND SECURITY

Preserve existing data and functionality.

Do not perform destructive database operations
without explicit authorization.

Implement appropriate input validation,
access control, secure data handling, and
protection against relevant security vulnerabilities.

10. EDGE CASES

Consider:
- Empty and invalid inputs
- Missing values
- Duplicate records
- Boundary conditions
- Database failures
- Network failures
- API errors
- Concurrent operations
- Unexpected user behaviour

Identify additional relevant edge cases.

11. IMPLEMENTATION REQUIREMENTS

Understand the existing project before modifying it.

Choose a reliable and maintainable approach.

Prefer minimal changes when working with existing code.

Avoid unnecessary dependencies and complexity.

Ensure that all components integrate correctly.

12. VALIDATION REQUIREMENTS

After implementation:

- Check syntax and imports.
- Verify the core logic.
- Test important functionality.
- Test relevant edge cases.
- Check database operations.
- Check integration between components.
- Review security risks.
- Check that existing functionality remains intact.

Fix identified defects and rerun affected tests.

Do not claim that a test passed unless it was
actually executed successfully.

13. SUCCESS CRITERIA

The task is complete only when:

1. All mandatory requirements are implemented.
2. The implementation behaves as specified.
3. Existing functionality and data are preserved.
4. Relevant tests have been performed.
5. Identified defects have been corrected.
6. Required files and dependencies are included.
7. Setup and execution instructions are provided.
8. Remaining limitations are disclosed.

14. OUTPUT FORMAT

```
Provide:

1. Brief implementation summary.
2. Final project structure.
3. Complete code for all required files.
4. Dependencies and installation instructions.
5. Database setup or migration instructions.
6. Execution instructions.
7. Testing performed and results.
8. Known limitations and unverified areas.

Do not omit required code or replace complete
implementations with unexplained placeholders.

# FINAL INSTRUCTION

Prioritize correctness, relevance, problem-solving,
efficiency, and final output quality.

Cross-check the implementation against the original
requirements before submitting it.

If a requirement is unclear and materially affects
correctness, ask for clarification rather than guessing.

Deliver the simplest complete solution that satisfies
the requirements without compromising reliability.
```

PART 11 — SPECIALIZED PROMPT TEMPLATES

The master template is suitable for complete development tasks. For smaller tasks, use a shorter specialized template.

A. Debugging an existing program

```
Act as a senior debugging engineer.

Analyze the supplied code and identify the root cause
of the reported issue.

Do not immediately rewrite the entire program.

1. Inspect the relevant code.
2. Understand the intended behaviour.
3. Trace the error to its underlying cause.
4. Identify the exact files and functions involved.
5. Apply the smallest reliable correction.
6. Preserve unrelated functionality.
7. Test the correction.
8. Check for regression issues.

Explain the cause, changes made, and validation results.

Do not claim that the issue is fixed unless the
```

```
relevant behaviour has been verified.
```

B. Adding a new feature

```
Act as a senior software engineer.

Implement the following feature in my existing project:

[Describe feature]

Before modifying the code:
- Inspect the existing architecture.
- Identify the relevant files and dependencies.
- Understand how the new feature interacts with
  existing functionality.

Requirements:
[List detailed requirements]

Preserve all existing functionality and data.

Modify only the files necessary for the feature.
Avoid unnecessary dependencies and unrelated changes.

Test the new feature and relevant existing functionality.

Provide the changed files, explanation, and test results.
```

C. Optimizing existing code

```
Act as a performance optimization engineer.

Analyze the supplied implementation and identify
genuine performance bottlenecks.

Evaluate:
- Time complexity
- Space complexity
- Database queries
- Memory usage
- Network operations
- Repeated calculations
- Unnecessary dependencies

Propose the most appropriate improvements.

Preserve the existing functionality, output, and
data behaviour.

Implement the improvements and compare the original
and optimized approaches where measurable.

Do not claim performance improvements without
supporting measurements or a clear technical basis.
```

D. Building a complete application

```
Act as a senior full-stack engineer and software architect.

Build a complete application based on the supplied
requirements and technical specifications.

First understand the problem, architecture, data flow,
user interactions, and expected workload.

Implement all required frontend, backend, database,
and integration components.

Ensure that:
- All features work together.
- Data is stored and retrieved correctly.
- Input validation is implemented.
- Errors are handled properly.
- Security is considered.
- The interface is usable and responsive.
- The application can be installed and executed.

Test the application wherever the environment allows.

Provide every required file, setup instruction,
dependency, and validation result.

Do not provide a partially implemented application
while describing it as complete.
```

E. Reviewing code without modifying it

```
Act as an independent senior software reviewer.

Review the supplied code without modifying it.

Identify:
- Functional defects
- Logical errors
- Security vulnerabilities
- Performance issues
- Data-integrity risks
- Compatibility problems
- Missing edge cases
- Maintainability problems

For every issue, provide:
1. File and location.
2. Description.
3. Severity.
4. Why it is a problem.
5. Recommended correction.

Separate confirmed defects from potential risks
and improvement suggestions.

Do not claim that you executed tests if you
only inspected the code.
```

PART 12 — FINAL QUALITY-CONTROL SCORECARD

Use this scorecard to evaluate prompts and their generated results.

Judging criterion	What to evaluate	Weight
Prompt Quality & Structure	Clarity, context, requirements, constraints	20%
Accuracy & Relevance	Technical correctness and requirement adherence	20%
Creativity	Useful innovation and practical improvements	10%
Problem Solving	Root-cause analysis, logic, edge cases	20%
Efficiency & Optimization	Performance, simplicity, resource usage	10%
Final Output Quality	Completeness, usability, testing, presentation	20%
Total		**100%**

Scoring guide

Score	Interpretation
0–2	Very poor
3–4	Below expectations
5–6	Acceptable, but needs improvement
7–8	Good
9	Excellent
10	Outstanding

Calculate the weighted score as:

Weighted score = Sum of (criterion score × criterion weight)

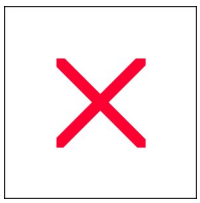
For example, if a criterion receives 8/10 and carries 20% weight, its contribution is 16 percentage points.

A high overall score must not hide a critical failure. A solution that loses existing user data, violates a mandatory restriction, or fails its central function should not be considered successful simply because it scores well in other categories.

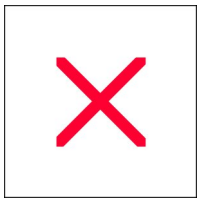
PART 13 — THE FINAL PRE-SUBMISSION CHECKLIST

Before submitting any important coding prompt, check the following.

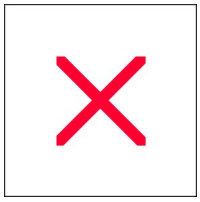
Prompt checklist

- 

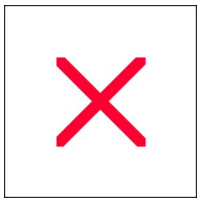
  Is the objective clearly defined?

- 

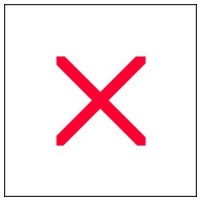
  Is the problem explained?

- 

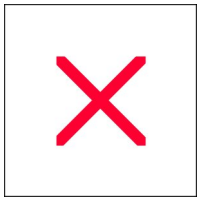
  Is sufficient context provided?

- 

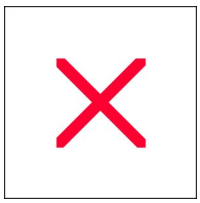
  Are all mandatory features listed?

- 

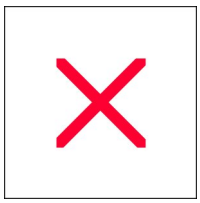
  Is the technology stack specified?

- 

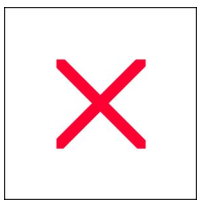
Are relevant versions specified?

- 

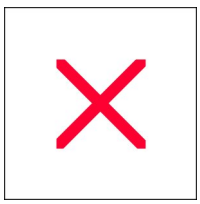
Are constraints and restrictions clear?

- 

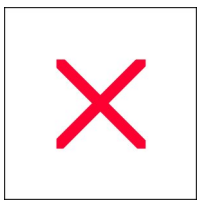
Are existing files and functionality explained?

- 

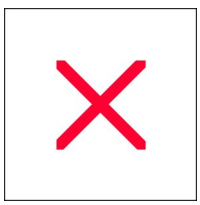
Are database requirements clear?

- 

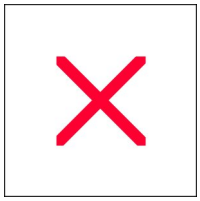
Are expected behaviours defined?

- 

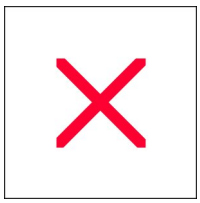
Are relevant edge cases included?

- 

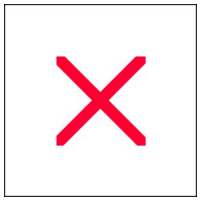
Are security requirements considered?

- 

  Are success criteria measurable?

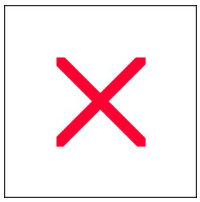
- 

  Is the expected output format specified?

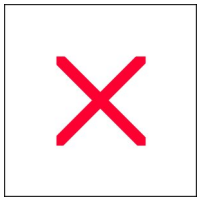
- 

  Are there any conflicting instructions?

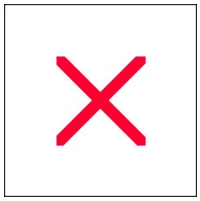
Code-quality checklist

- 

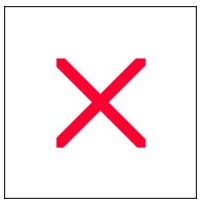
  Does the implementation solve the actual problem?

- 

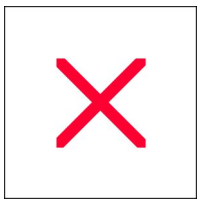
  Are all mandatory requirements implemented?

- 

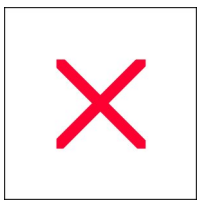
Is the code syntactically valid?

- 

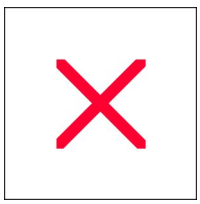
Is the logic correct?

- 

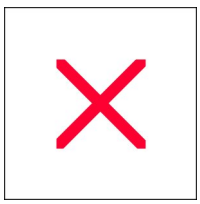
Are components properly integrated?

- 

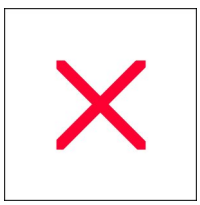
Are invalid inputs handled?

- 

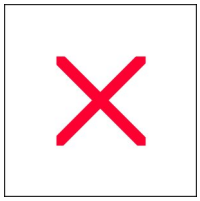
Are relevant edge cases tested?

- 

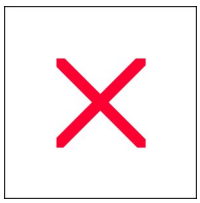
Is existing functionality preserved?

- 

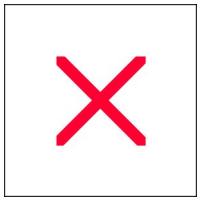
Is existing data protected?

- 

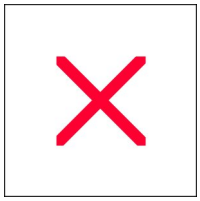
  Are security risks addressed?

- 

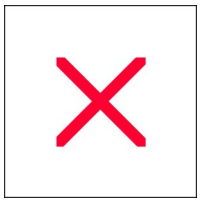
  Are unnecessary dependencies avoided?

- 

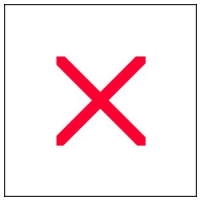
  Is the solution reasonably optimized?

- 

  Are installation and execution instructions complete?

- 

  Are test results reported honestly?

- 

  Are limitations disclosed?

PART 14 — THE 25 GOLDEN RULES

1. Define the outcome before requesting implementation.
2. Explain the problem, not just the desired code.
3. Provide sufficient project context.
4. Specify the exact programming environment.
5. Clearly distinguish mandatory requirements from preferences.
6. Use structured prompts.
7. Avoid ambiguous instructions.
8. Avoid contradictory requirements.
9. Use examples for complicated behaviour.
10. Define expected input and output.
11. Specify database and API requirements.
12. Consider security from the beginning.
13. Consider realistic edge cases.
14. Define measurable success criteria.
15. Inspect existing code before modifying it.
16. Preserve existing functionality.
17. Protect existing data.
18. Avoid unnecessary rewrites.
19. Avoid unnecessary dependencies.
20. Prefer simple and maintainable solutions.
21. Validate code using actual tools wherever possible.
22. Fix defects and retest.
23. Check integration, not just individual files.
24. Never accept fabricated test results.
25. Cross-check the final output against the original requirements.

FINAL PRINCIPLE

A perfect prompt is not the longest prompt. It is the prompt that communicates the right information, establishes the correct boundaries, defines measurable success, and enables the AI to produce a solution that can be verified.

The ultimate objective is not to make GPT-5.6 Luna or Gemini 3.8 Flash generate more code. It is to make them generate the **right code, for the right problem, using the right approach, with the fewest avoidable mistakes.**

Always prioritize:

Understand → Plan → Implement → Integrate → Test → Verify → Deliver.

A solution should be considered complete only when its actual behaviour has been checked against the requirements—not merely when code generation has finished.